

Brasaland Backend Architecture Proposal

Readable summary — full technical detail lives in [ARCHITECTURE PROPOSAL.md](#) .

For: CTO / Brasaland Digital

About: How we should structure the FastAPI backend before the first endpoints

Company: Brasaland — 14 restaurants in Colombia and Florida (USD and COP)

What this document is for

The CTO asked for reasoning, not code: which pattern to use, how to organize modules and routes, how the frontends and backend stay separate, and what can go wrong.

In short: this is documented technical reasoning, not a technology shopping list.

What Brasaland needs from a backend

Brasaland is a grilled food chain with two countries, two currencies, and messy day-to-day data (WhatsApp orders, weak inventory visibility, incomplete loyalty and hiring data).

We already have:

- A public site (`uis/website`) for brand and Brasa Points
- An internal backoffice (`uis/backoffice`) for operations metrics
- Milestone 2 TypeScript logic under root `src/` (sales, waste, margins, rankings)

So the first API should serve **Restaurant Operations**. Brasa Points and People & Talent are named as later domains, not fully designed yet.

All company APIs must live under `services/` . The UIs must not become the API.

Pattern we chose (and why)

Modular layered architecture, organized by business domain.

Layers in practice:

1. **Routers + schemas** — HTTP and request/response shapes (FastAPI + Pydantic)
2. **Services** — Python business rules (ported Milestone 2 formulas)
3. **Contracts** — shared naming so TypeScript `src/` and Python stay aligned
4. **App factory** — one FastAPI app that mounts each domain router

Why this fits Brasaland:

- Felipe needs one trusted place for margin, waste, and location scores
- Website and backoffice both need one company API
- USD/COP rules should not be copy-pasted into React
- Later milestones (loyalty, hiring) can add domains without one giant routes file
- The monorepo already asks for one centralized backend, not many microservices

What we are not doing in v1: classic MVC as the main story, serverless-first, or many microservices.

What FastAPI research told us

Standard guidance (domain packages, thin routers, `service.py` , shared `core` config, `include_router`) comes from:

- <https://github.com/zhanymkanov/fastapi-best-practices>
- <https://www.dsinnovators.com/blog/python/fastapi-production-scalable-apis-2024/>
- <https://fastapi.tiangolo.com/tutorial/bigger-applications/>
- <https://fastapi.tiangolo.com/tutorial/cors/>

That is why Brasaland's API is proposed as `services/api` with folders like `operations/` , `loyalty/` , and `talent/` , each holding router + schemas + service.

Proposed folder layout

```
services/api/
  app/
    main.py          # create app, CORS, include routers
    core/            # config, CORS helpers, shared errors
    contracts/       # names/shapes aligned with root src/
    operations/      # v1 priority
    loyalty/         # later (Brasa Points)
    talent/          # later (People & Talent)
  tests/
```

We split domains when the **business owner, vocabulary, or audience** changes:

- **operations** — Felipe / location managers (internal metrics)
- **loyalty** — marketing / customers (Brasa Points)
- **talent** — People & Talent (candidates later)

Milestone 2 TypeScript vs FastAPI

- Keep pure logic in root `src/` as the reference.
 - **Port** the same formulas into Python `operations/service.py` .
 - Keep a shared **contract** for field names and meanings.
 - Do not rewrite formulas inside React, and do not invent different names per language.
-

How routes are grouped

One FastAPI app. Several routers. Versioned under `/api/v1` .

Public (v1)

- GET `/health`
- Optional GET `/api/v1/meta`

Operations (v1 priority, internal-intended) — prefix `/api/v1/operations`

- Locations list/detail
- Margin, waste cost, rankings
- Sales summary, top items, country comparison
- Validation endpoints for menu, sale, location
- Financial routes take `currency=USD | COP`

Loyalty (sketch later) — `/api/v1/loyalty`

- Brasa Points registration

Talent (sketch later) — `/api/v1/talent`

- Candidates and notes

Auth is **out of scope for v1**, but operations and talent stay under clear prefixes so auth can wrap them later. Persistence / database choice is **not** part of this proposal.

Frontends and backend as separate systems

We stay in the **monorepo**, but they still run as separate systems:

- Frontends: Next.js in `uis/website` and `uis/backoffice`
- Backend: FastAPI in `services/api`
- They talk only over **HTTP JSON**
- API base URL and CORS origins come from **environment variables**
- No Next.js route handlers as a substitute for `services/`

Without CORS and correct env vars, the browser can look “broken” even when the API is healthy.

Decisions at a glance

Topic	Decision
Pattern	Modular layered + domain packages
Framework	FastAPI
Layout	Single app at <code>services/api</code>
First domain	Operations
Milestone 2	Keep TS in <code>src/</code> ; port to Python
Auth	Later (route groups prepared now)
Database	Not specified here
FE/BE	Monorepo, separate processes, HTTP + env + CORS

Risks if we ignore this structure

1. **TS and Python formulas drift** — backoffice and API show different numbers.
 2. **APIs hidden inside Next.js** — breaks the monorepo rule and duplicates logic.
 3. **CORS / env mistakes** — look like frontend bugs.
 4. **All routes treated as public** — operations and talent data get exposed later.
 5. **Too many microservices too early** — extra deploy and CORS pain while learning FastAPI.
-

Bottom line for the next sprint

Agree on this shape before coding endpoints:

1. One modular FastAPI app under `services/api` .

2. Operations first; loyalty and talent placeholders.
3. Thin routers; Python services own ported Milestone 2 rules.
4. TypeScript `src/` stays the reference for pure logic.
5. UIs call the API over HTTP with env + CORS.
6. Auth and database come later.

For the complete tables, endpoint list, and grading-oriented detail, read [ARCHITECTURE_PROPOSAL.md](#) .